

Harvard John A. Paulson
School of Engineering
and Applied Sciences



Kempner
INSTITUTE
at Harvard University

Train, Reason, and Act

Under Value Function Guidance

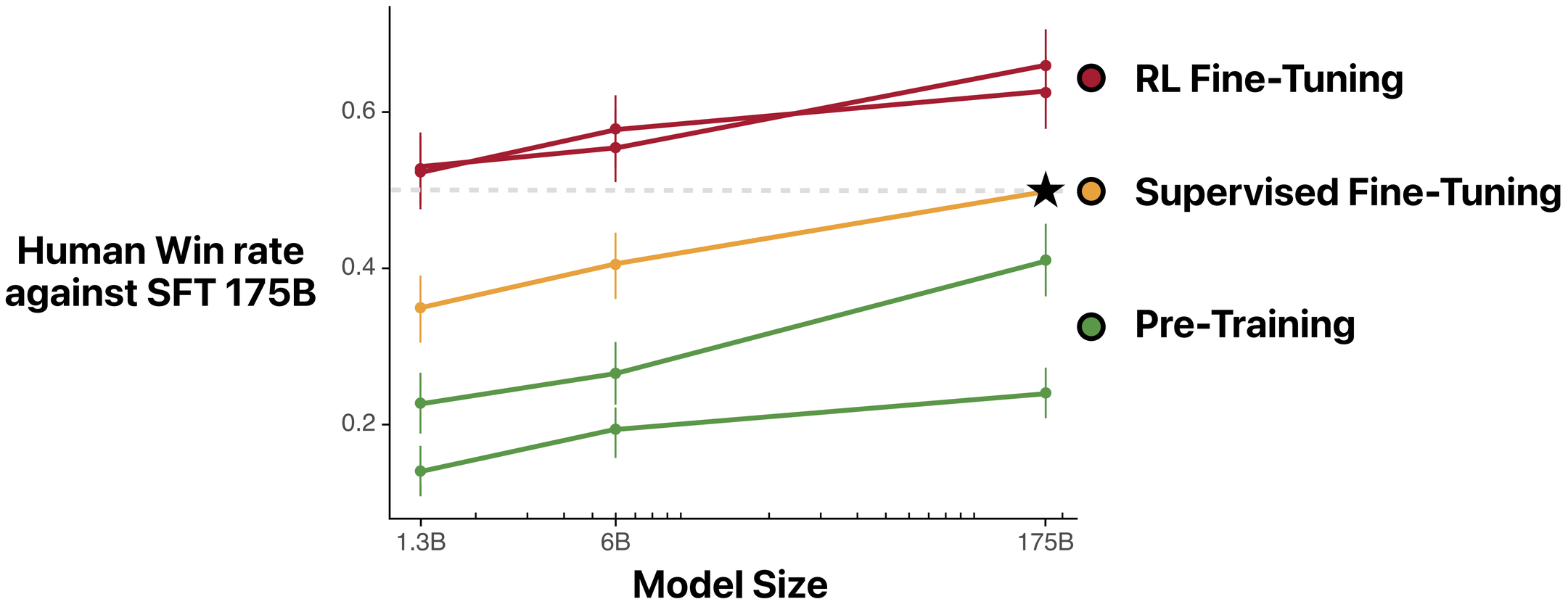
Kianté Brantley, Assistant Professor, Harvard University

"A large language model is a deep learning model trained on massive text datasets to understand and generate human language."

Large Language Model

How can value functions make LLMs more efficient at training, reasoning, and acting?

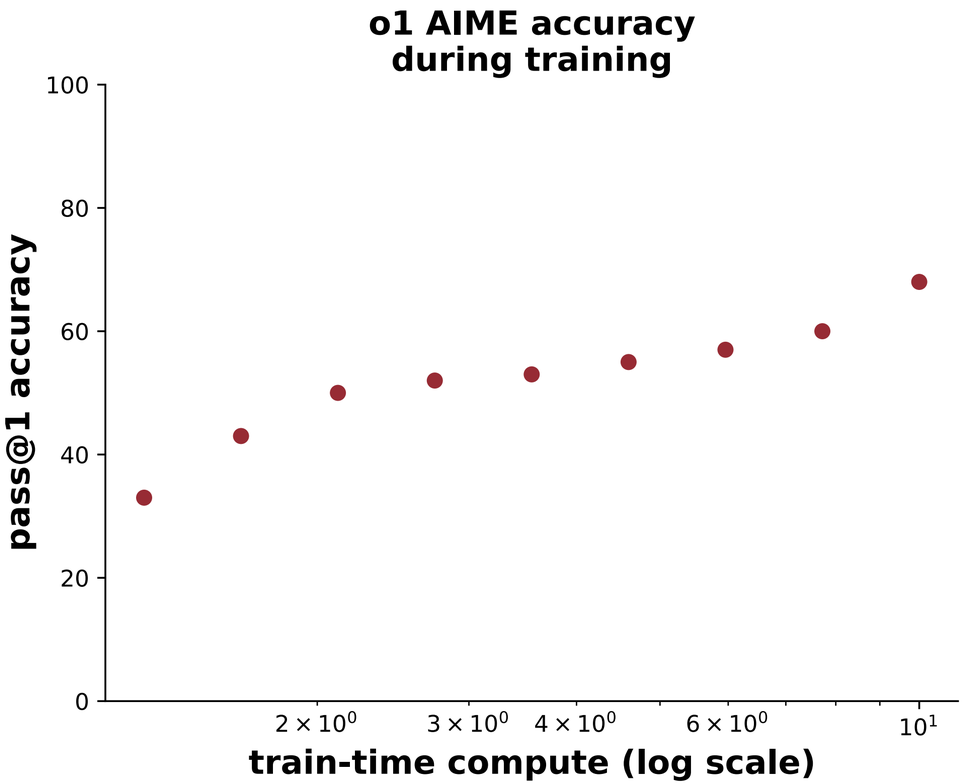
Learning from Human Feedback

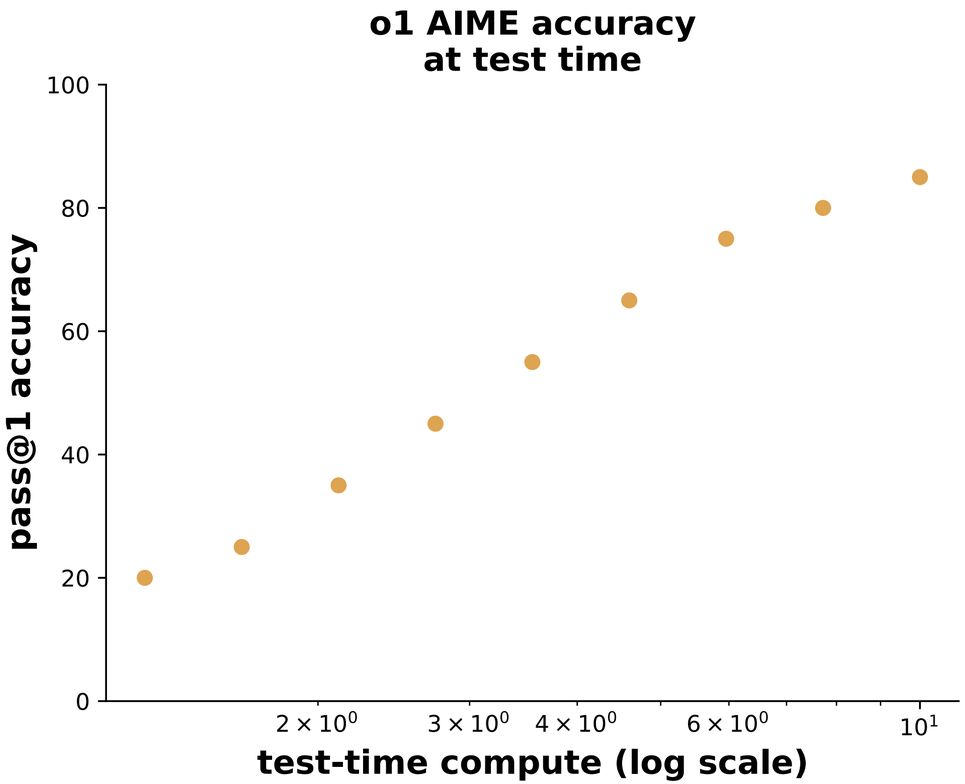


Long Ouyang et al. Training language models to follow instructions with human feedback OpenAI 2022

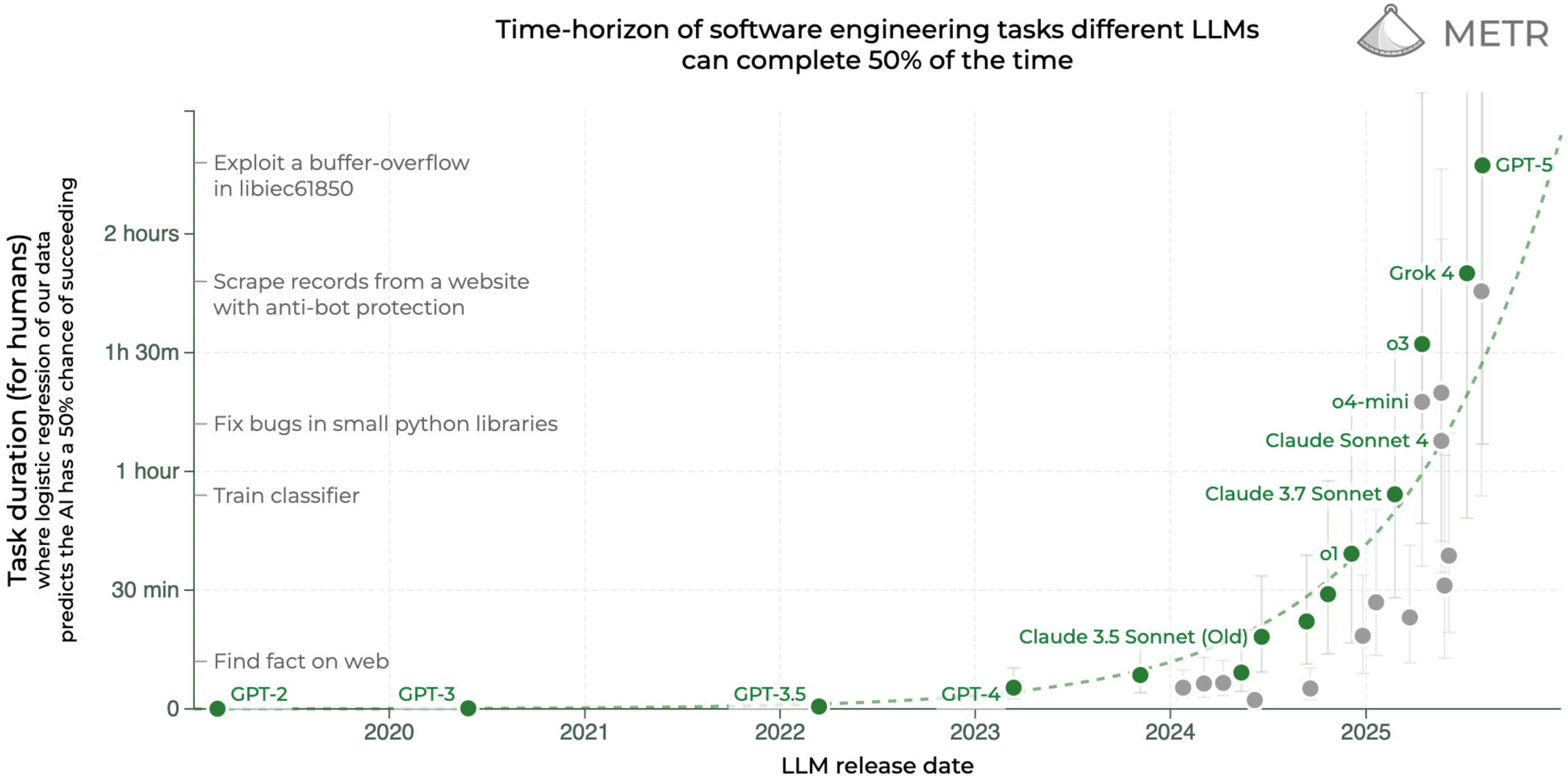
Learning to reason with LLMs

“We are introducing OpenAI o1, a new large language model trained with reinforcement learning to perform complex reasoning.” - OpenAI



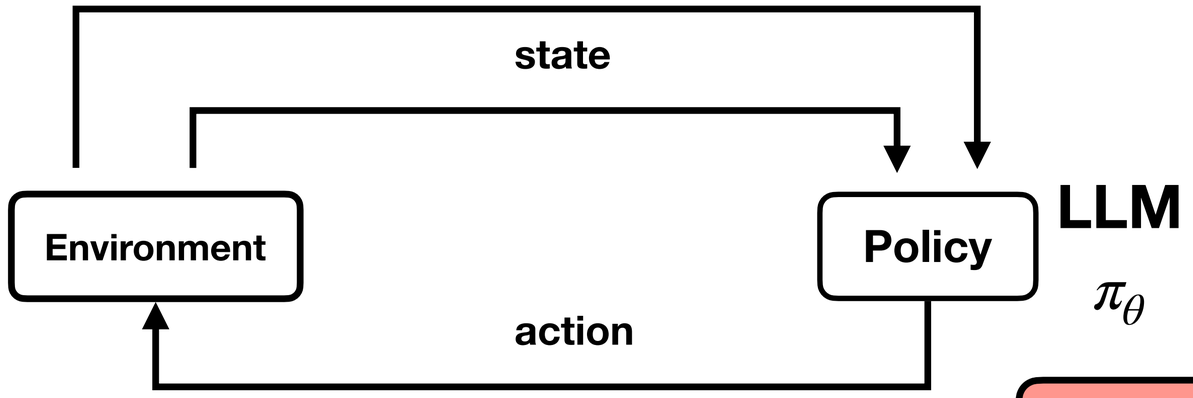


Learning to reason with LLMs



LLMs Post-Training MDP

reward (Verifiable tasks,e.g. math, or human preference)



How do we optimize π_θ ?

Prompt x:

What is 1+1?

Action y:

<think>
1 + 1 is a simple arithmetic addition.
Adding one to one equals two.
Therefore, the answer is 2.
</think>
$\boxed{2}$

LLMs Post-Training MDP

reward (Verifiable tasks,e.g. math, or human preference)



$$J(\pi) = \underbrace{\mathbb{E}_{\pi}\left[r(x, y)\right]}_{\text{Maximize reward}} - \frac{1}{\eta} \underbrace{\mathbb{D}_{\mathrm{KL}}(\pi \parallel \pi_{\mathrm{ref}})}_{\text{Minimal deviation}}$$

under

LLMs Post-Training

policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, r(s,a)\right]$$

Most LLM post-training work focuses on **stabilizing policy-gradient methods.**

Can the structure of LLM generation be leveraged for greater efficiency?

$$J(\pi) = \underbrace{\mathbb{E}_\pi\left[r(x,y)\right]}_{\text{Maximize reward}} - \frac{1}{\eta}\underbrace{\mathbb{D}_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})}_{\text{Minimal deviation}}$$

under

LLMs Post-Training

policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, r(s,a)\right]$$

Environment

Policy

LLMs implicitly execute a token-by-token MDP with a special structure.

Prompt x:

What is 1+1?

Action y:

<think>
1 + 1 is a simple arithmetic addition.
Adding one to one equals two.
Therefore, the answer is 2.
</think>
$\boxed{2}$

LLMs Post-Training

policy gradient

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s)\, r(s,a)\right]$$

Environment

Policy

LLMs implicitly execute a token-by-token MDP with a special structure.

$$J(\pi) = \mathbb{E}_{\pi}\left[r(x,y)\right] - \frac{1}{\eta}\,\mathbb{D}_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})$$

Maximize reward under Minimal deviation

How can we take advantage of this special structure?

LLMs implicitly execute a token-by-token MDP with a special structure.

What does the token-by-token structure of LLMs buy us?

Optimal Value *Function Guidance*

I. **Q#:** Principled value-based algorithm for LLM post — Act

II. **VGS:** Test-time Value Guided Search — Reason

III. **A*PO**: Training-time optimization algorithm — Train

LLMs implicitly execute a token-by-token MDP with a special structure.

What does the token-by-token structure of LLMs buy us?

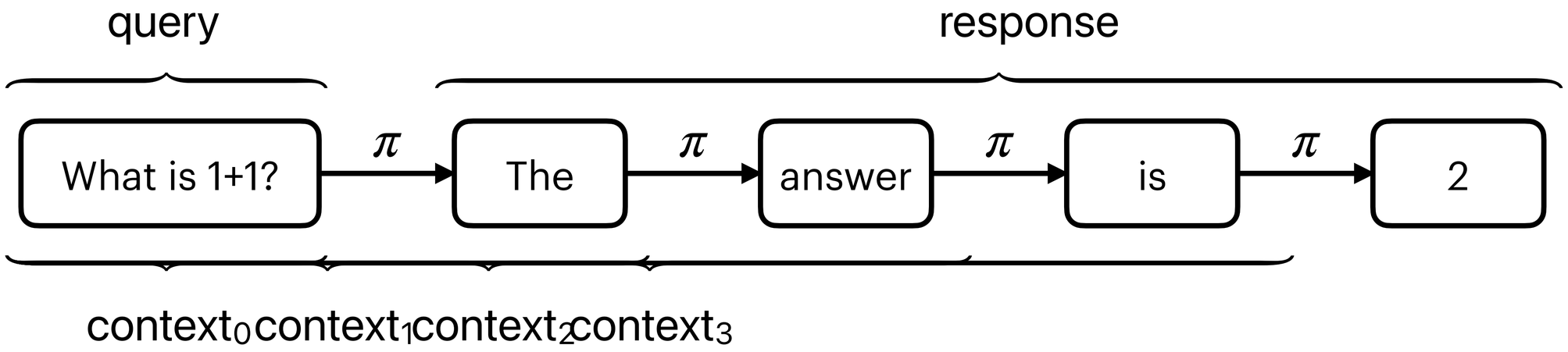
I. **Q#:** Principled value-based algorithm for LLM post-training

Q#: Provably Optimal Distributional RL for LLM Post-Training

Jin Peng Zhou , Kaiwen Wang , Jonathan Chang , Zhaolin Gao,
Nathan Kallus, Kilian Q. Weinberger, Kianté Brantley, and Wen Sun

Cornell University, Harvard University, Netflix, Databricks

Autoregressive Policies



Let y_0 = query **and** y_h = [query, context$_{1:h-1}$] $\triangleq y_{0:h-1} = (y_0, y_1, \ldots, y_{h-1})$

$$V^{\pi}(\text{context}_{0:h-1}) = \mathbb{E}_{\pi}[r(\text{context}_{0:h-1}, \cdot) - \eta \text{KL}(\pi(\cdot \mid \text{context}_{0:h-1}), \pi^{\text{ref}}(\cdot \mid \text{context}_{0:h-1}))]$$

$$V^{\pi}(y_h) = \mathbb{E}_{\pi}[r(y_h, \cdot) - \eta \text{KL}(\pi(\cdot \mid y_h), \pi^{\text{ref}}(\cdot \mid y_h)]$$

Autoregressive Policies

Let $\pi^{\#}$ denote the optimal policy **then** $\pi^{\#}(\,\cdot \mid y_h) \propto \pi_{\text{ref}}(\,\cdot \mid y_h)\, \exp(\eta Q^{\#}(\,\cdot\,, y_h))$

$$V^{\#}(y_h) = \frac{1}{\eta} \log \mathbb{E}_{\pi_{\text{ref}}}[\exp(\eta Q^{\#}(\,\cdot\,, y_h))]$$

Optimal Value Function

$$\exp(\eta V^{\#}(y_h)) = \mathbb{E}_{\pi_{\text{ref}}}[\exp(\eta Q^{\#}(x, y))] \quad \textbf{and} \quad Q^{\#}(\,\cdot\,, y_h) = r(\,\cdot\,, y_h) + \mathbb{E}_{x' \sim P(\cdot, y_h)}\left[V^{\#}(y_{h+1})\right]$$

$$\exp(\eta V^{\#}(y_h)) = \mathbb{E}_{\pi^{\text{ref}}}[\exp(\eta \sum_{h' \geq h} r_t(\,\cdot\,, y_{h'})) \,|\, y_h]$$

Deterministic Trans.

$$\exp(\eta Q^{\#}(\,\cdot\,, y_h)) = \mathbb{E}_{\pi^{\text{ref}}}[\exp(\eta \sum_{h' \geq h} r_t(\,\cdot\,, y_{h'})) \,|\, \cdot\,, y_h]$$

Optimal Q-function

Autoregressive Policies

Let $\pi^{\#}$ denote the optimal policy **then** $\pi^{\#}(\cdot \mid y_h) \propto \pi_{\text{ref}}(\cdot \mid y_h) \exp(\eta Q^{\#}(\cdot, y_h))$

$$V^{\#}(y_h) = \frac{1}{\eta} \log \mathbb{E}_{\pi_{\text{ref}}}[\exp(\eta Q^{\#}(\cdot, y_h))]$$

Optimal Value Function

$$\exp(\eta V^{\#}(y_h)) = \mathbb{E}_{\pi_{\text{ref}}}[\exp(\eta Q^{\#}(x, y))] \quad \textbf{and} \quad Q^{\#}(\cdot, y_h) = r(\cdot, y_h) + \mathbb{E}_{x' \sim P(\cdot, y_h)}\left[V^{\#}(y_{h+1})\right]$$

$$\exp(\eta V^{\#}(y_h)) = \mathbb{E}_{\pi^{\text{ref}}}[\exp(\eta \sum_{h' \geq h} r_t(\cdot, y_{h'})) \,|\, y_h]$$

Deterministic Trans.

$$\exp(\eta Q^{\#}(\cdot, y_h)) = \mathbb{E}_{\pi^{\text{ref}}}[\exp(\eta \sum_{h' \geq h} r_t(\cdot, y_{h'})) \,|\, \cdot, y_h]$$

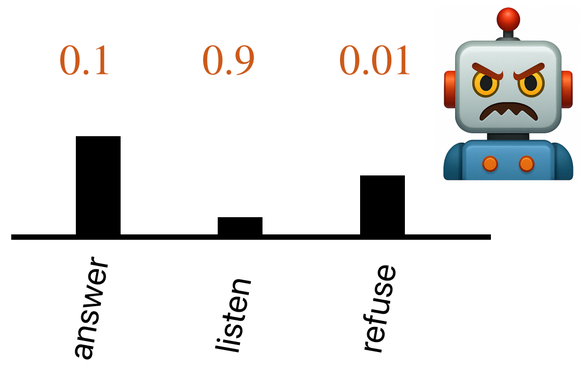
Optimal Q-function

Autoregressive Policies

Assume $\text{context}_{1:h}$ = "A helpful assistant should"

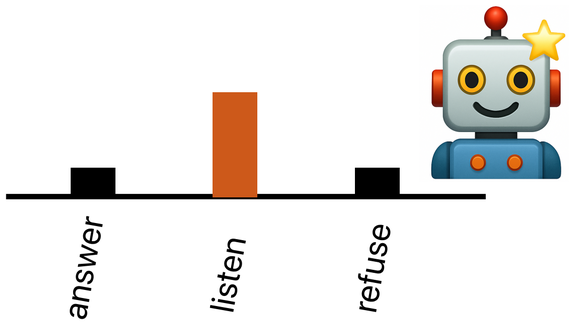
$$\exp(\eta Q^{\#}(\text{context}_{1:h})) = \mathbb{E}_{\pi^{\text{ref}}}[\exp(\eta \sum_{h' \geq h} r_t(\cdot, \text{context}_{1:h})) \mid \cdot, \text{context}_{1:h}]$$

$Q^{\#}(\cdot, [\text{"A helpful assistant should"}])$ →



$\pi^{\text{ref}}(\cdot \mid \text{"A helpful assistant should"})$ →

$\pi_{\text{ref}}(\cdot \mid \cdot)\ \exp(\eta Q^{\#}(\cdot, \cdot))$ →

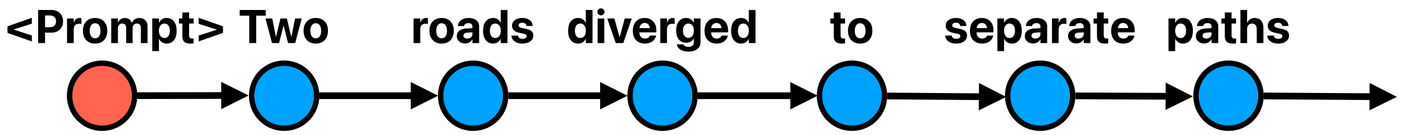


Algorithm: Q#

Step 1:

Sample a prompt from $y_0 \sim D$

Sample a generation from $\hat{\pi} \propto \pi^{\text{ref}}(\cdot \mid y_h)\left((1-\hat{p}(y_h, \cdot))e^0 + \hat{p}(y_h, \cdot)e^{\frac{1}{\eta}}\right)$



Algorithm: Q#

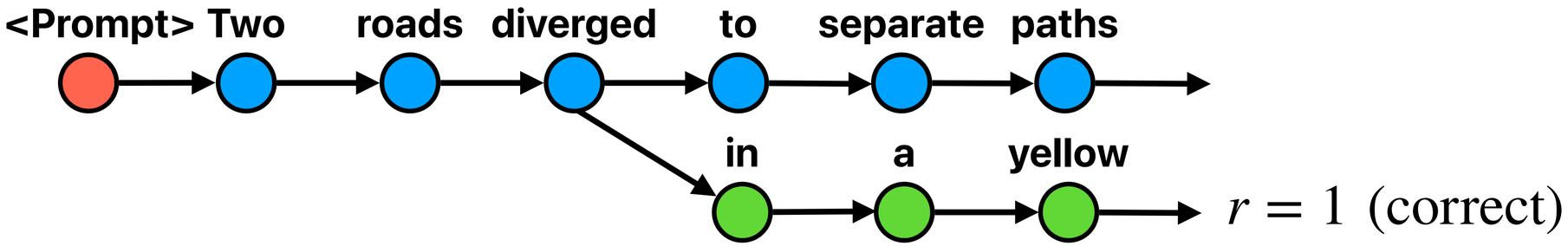
Transition: $P(s' \mid s, a)$ Deterministic

Step 1:

Sample a prompt from $y_0 \sim D$

Sample a generation from $\hat{\pi} \propto \pi^{\text{ref}}(\cdot \mid y_h)\left((1 - \hat{p}(y_h, \cdot))e^0 + \hat{p}(y_h, \cdot)e^{\frac{1}{\eta}}\right)$

Reset and sample a generation from π^{ref}



Algorithm: Q#

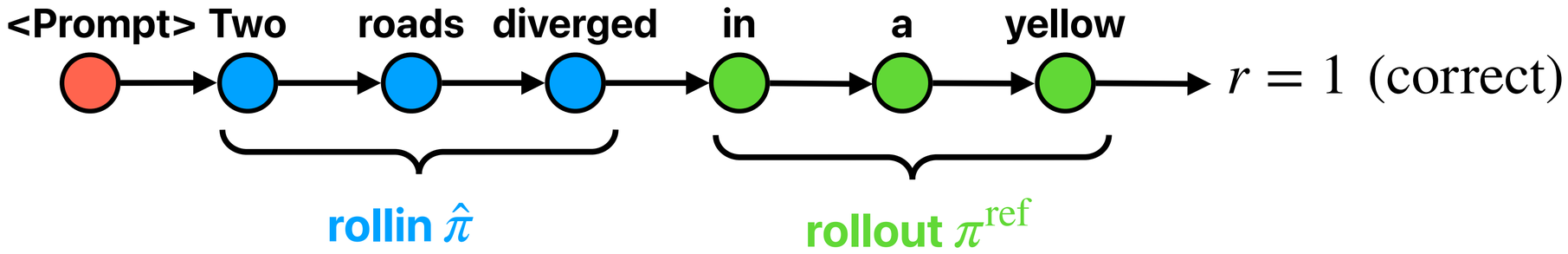
Transition: $P(s' \mid s, a)$ Deterministic

Step 1:

Sample a prompt from $y_0 \sim D$

Sample a generation from $\hat{\pi} \propto \pi^{\text{ref}}(\cdot \mid y_h)\left((1 - \hat{p}(y_h, \cdot))e^0 + \hat{p}(y_h, \cdot)e^{\frac{1}{\eta}}\right)$

Reset and sample a generation from π^{ref}



Algorithm: Q#

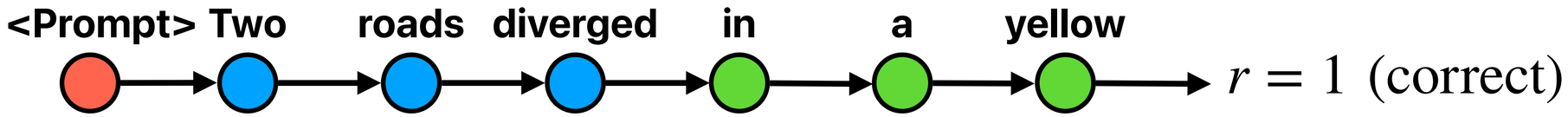
Transition: $P(s' \mid s, a)$ Deterministic

Step 1:

Sample a prompt from $y_0 \sim D$

Sample a generation from $\hat{\pi} \propto \pi^{\text{ref}}(\cdot \mid y_h)\left((1-\hat{p}(y_h, \cdot))e^0 + \hat{p}(y_h, \cdot)e^{\frac{1}{\eta}}\right)$

Reset and sample a generation from π^{ref}



Step 2:

Learn $\hat{p} \in \arg\min_p L_{\text{BCE}}(r, p)$

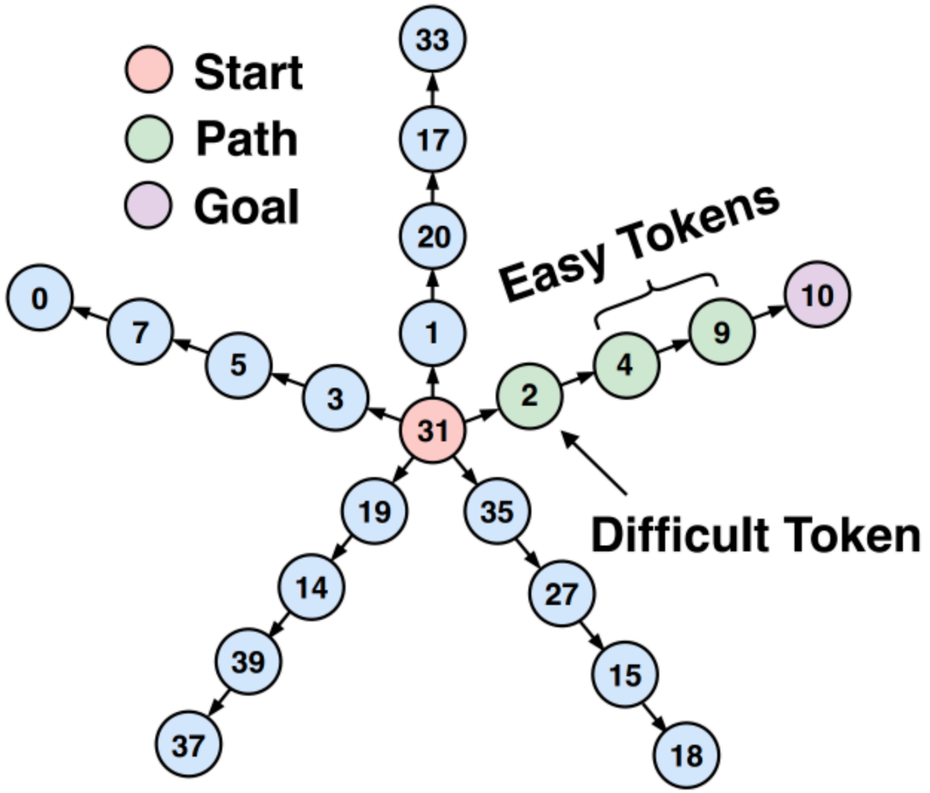
Step 3:

Set $\hat{\pi}(\cdot \mid y_h) \propto \hat{\pi} \propto \pi^{\text{ref}}(\cdot \mid y_h)\left((1-\hat{p}(y_h, \cdot))e^0 + \hat{p}(y_h, \cdot)e^{\frac{1}{\eta}}\right)$

Repeat

Note: learning $\hat{p}$ by classification instead of learning $\hat{Q}$ by regression!

Experiments: The Star Graph Experiment



Given:

(start, goal)= (31,10) and edge set

Easy Lookahead Task:

p=(start, goal, edge set)

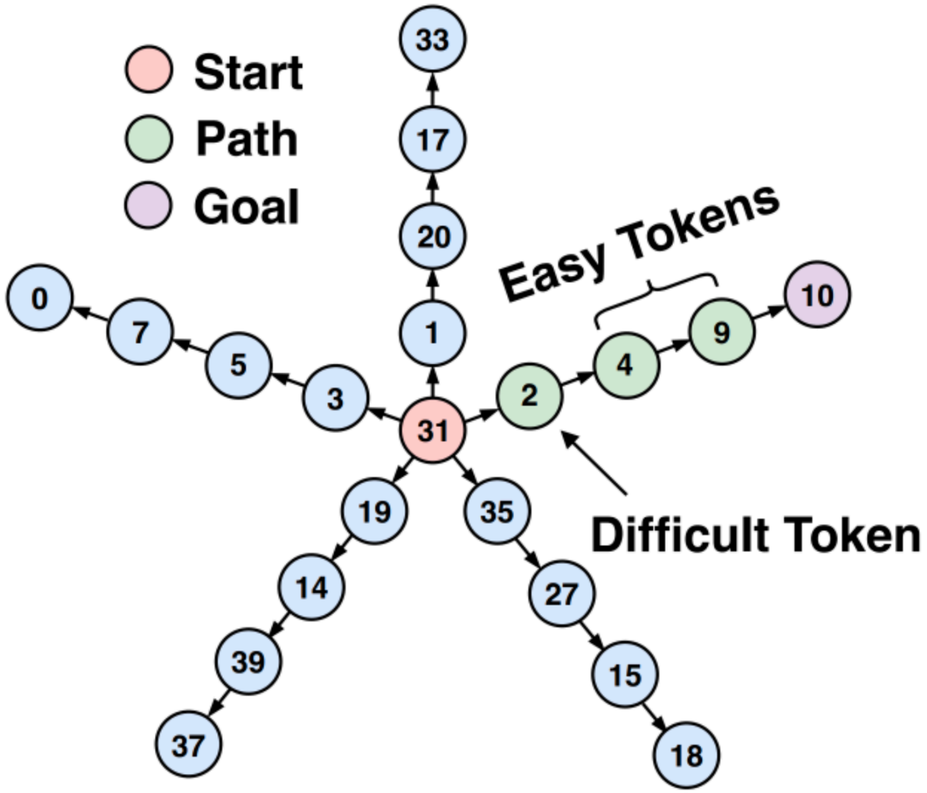
Learning goal:

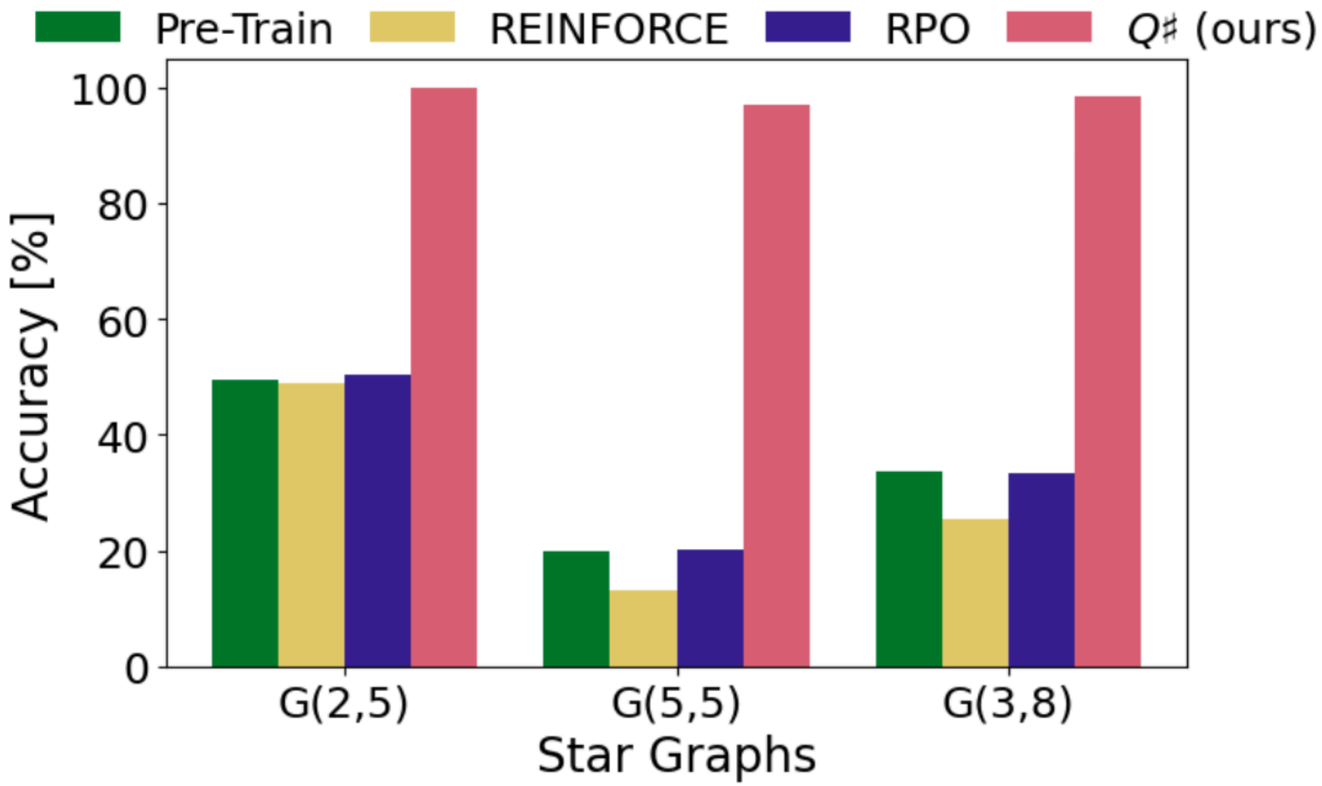
map $p \rightarrow r$ **where** r = (31, 2, 4, 9, 10)

Teacher-forcing

map $(p, r_{<i}) \rightarrow r$

Experiments: The Star Graph Experiment





Empirics: Math Reasoning

GSM8k:

Grade school-level math problems.

Model Types:

Q# model = Llama3 **1B**
with a scoring head, $\eta = 0.1$

π^{ref}	Llama 3 8B			Llama 3.1 8B		
Methods	π^{ref}	CD	$Q\sharp$	π^{ref}	CD	$Q\sharp$
`pass@1` ↑	69.1	77.8	**78.4**	82.9	84.5	**85.1**
`maj1@8` ↑	85.8	87.2	**88.1**	90.5	90.9	**91.4**
KL-Divergence ↓	-	6.39	**2.65**	-	7.43	**3.67**

Empirics: Math Reasoning

Math:

challenging problems from high school math competitions.

Model Types:

Q# model = Llama3 **1B**

with a scoring head, $\eta = 0.1$

π^{ref}	Llama 3 8B			Llama 3.1 8B		
Methods	π^{ref}	CD	$Q\sharp$	π^{ref}	CD	$Q\sharp$
`pass@1` ↑	25.4	24.9	**27.1**	43.9	45.3	**46.7**
`maj1@8` ↑	34.3	34.3	**37.9**	57.0	59.0	**60.1**
KL-Divergence ↓	-	15.27	**7.14**	-	26.8	**8.69**

Empirics: Math Reasoning

Math:

challenging problems from high school math competitions.

Model Types:

Q# model = Llama3 **1B**

with a scoring head, $\eta = 0.1$

π^{ref}	Llama 3.1 70B		
$Q\sharp$ Model	None	Llama 3.2 1B	Llama 3.2 3B
`pass@1` ↑	60.6	63.1	**64.1**
`maj1@8` ↑	69.0	72.5	**72.7**
KL-Divergence ↓	-	**4.98**	4.99

LLMs implicitly execute a token-by-token MDP with a special structure.

What does the token-by-token structure of LLMs buy us?

I. **Q#:** Principled value-based algorithm for LLM post-training

II. **VGS:** Test-time Value Guided Search

Value-Guided Search for Efficient Chain-of-Thought Reasoning

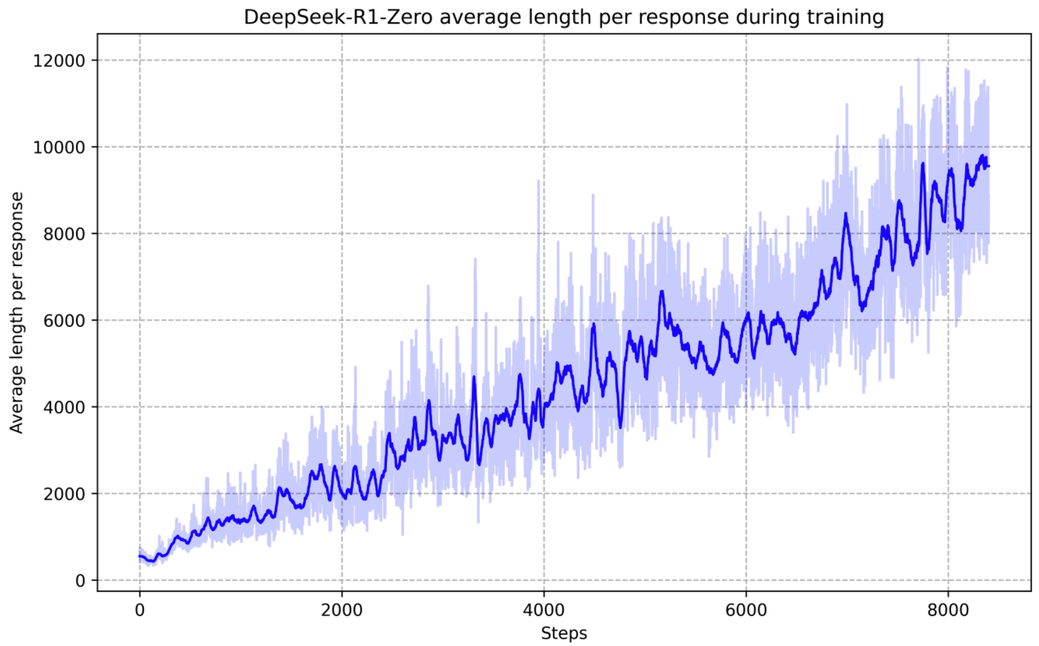
Kaiwen Wang, Jin Peng Zhou, Jonathan Chang,
Zhaolin Gao, Nathan Kallus, Kianté Brantley, and Wen Sun

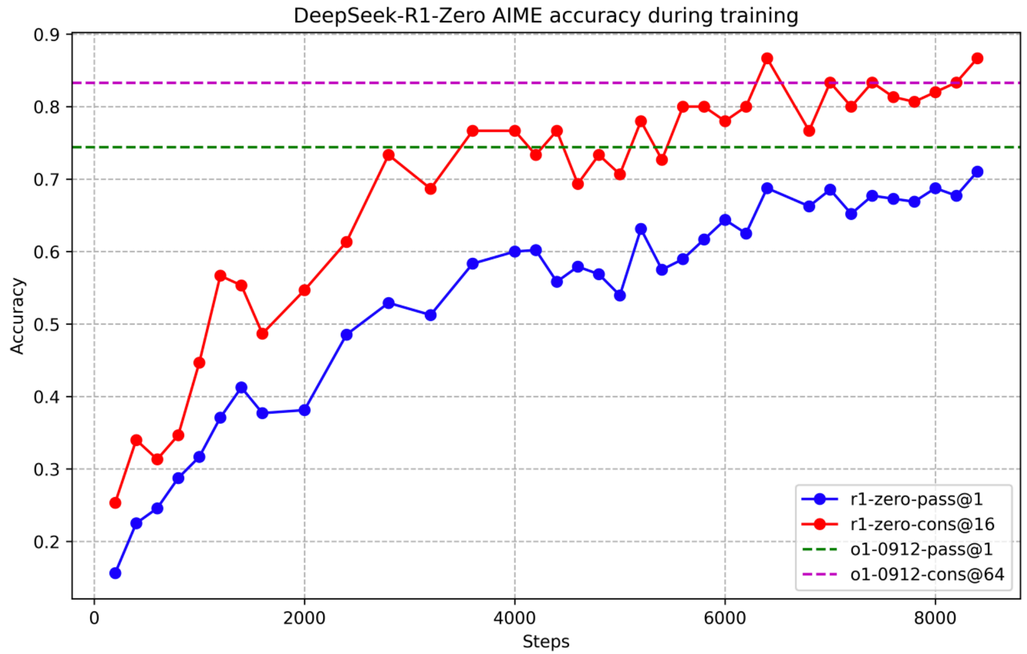
Cornell University, Harvard University, Netflix, Databricks

Learning to reason with LLMs

DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

"We introduce our first-generation reasoning models ..... trained via large-scale reinforcement learning (RL) without supervised fine-tuning (SFT) as a preliminary step"





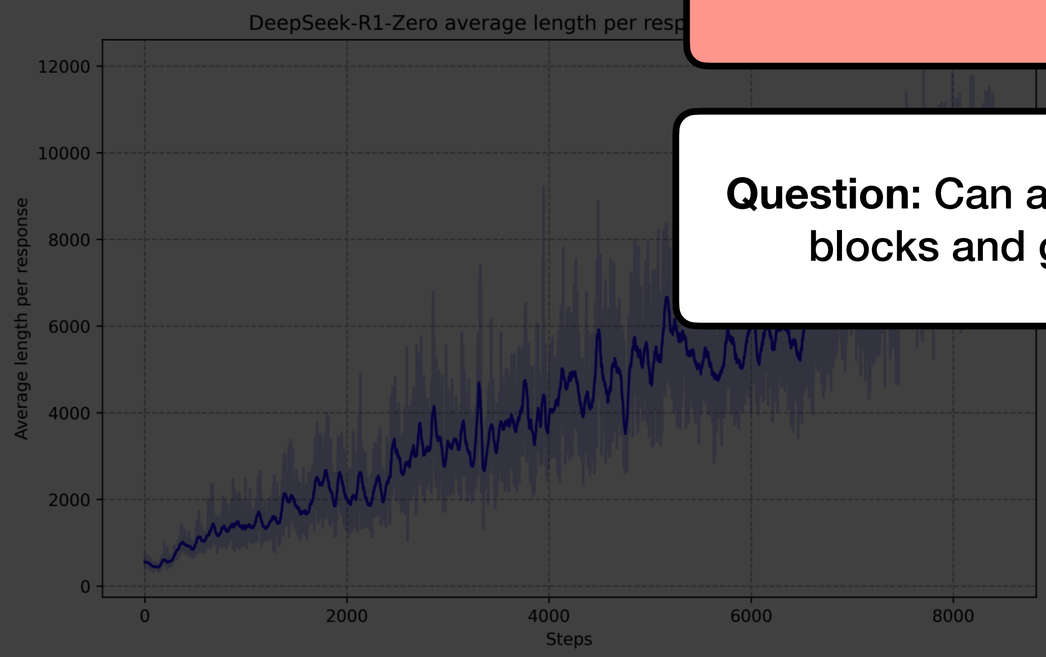
Learning to reason with LLMs

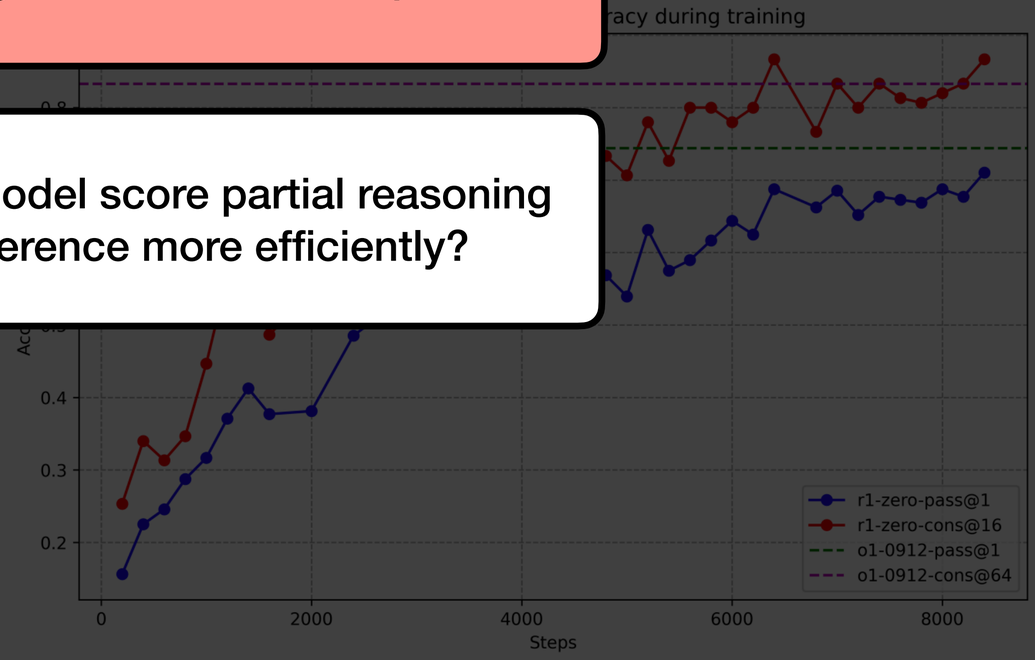
DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning

"We introduce our first-ge ment learning (RL)
w

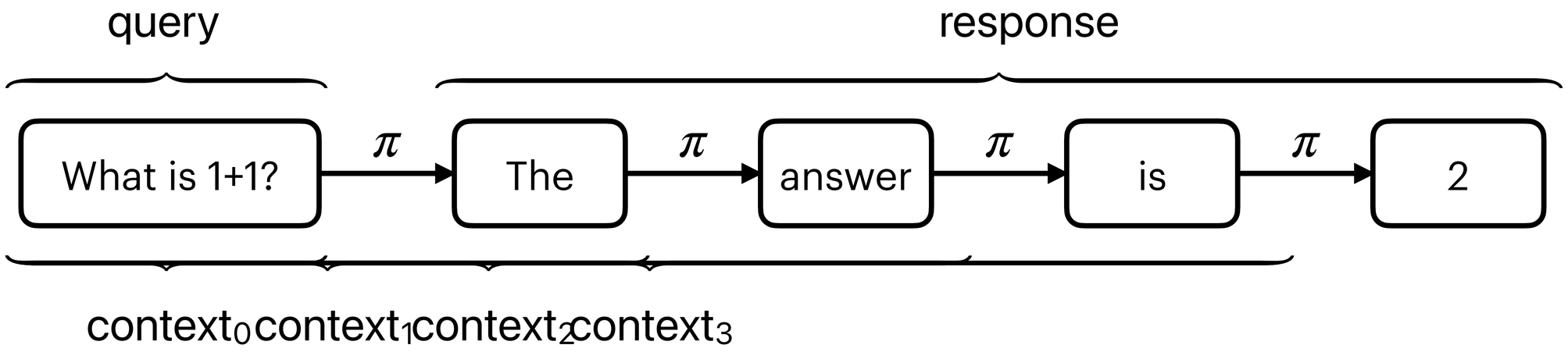
Problem: Best-of-N and majority voting only use value information after full generations are complete.

Question: Can a value model score partial reasoning blocks and guide inference more efficiently?

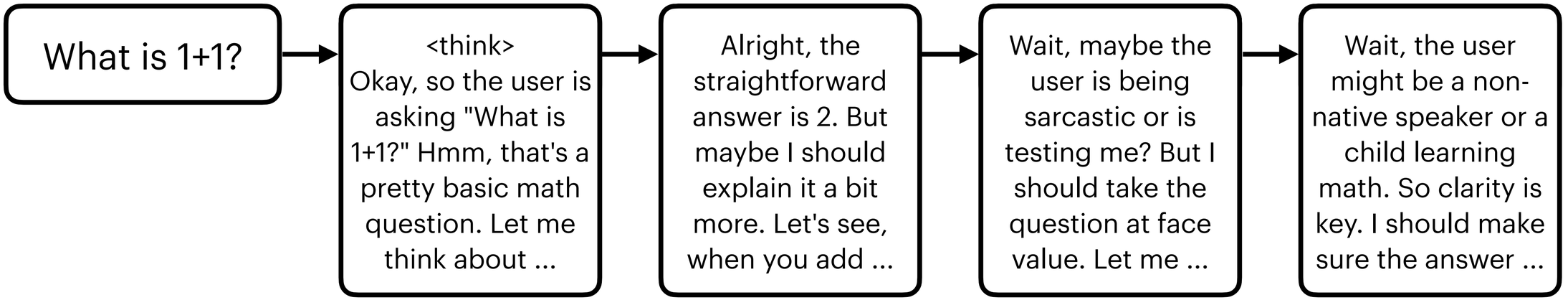




Autoregressive Policies



From tokens to blocks

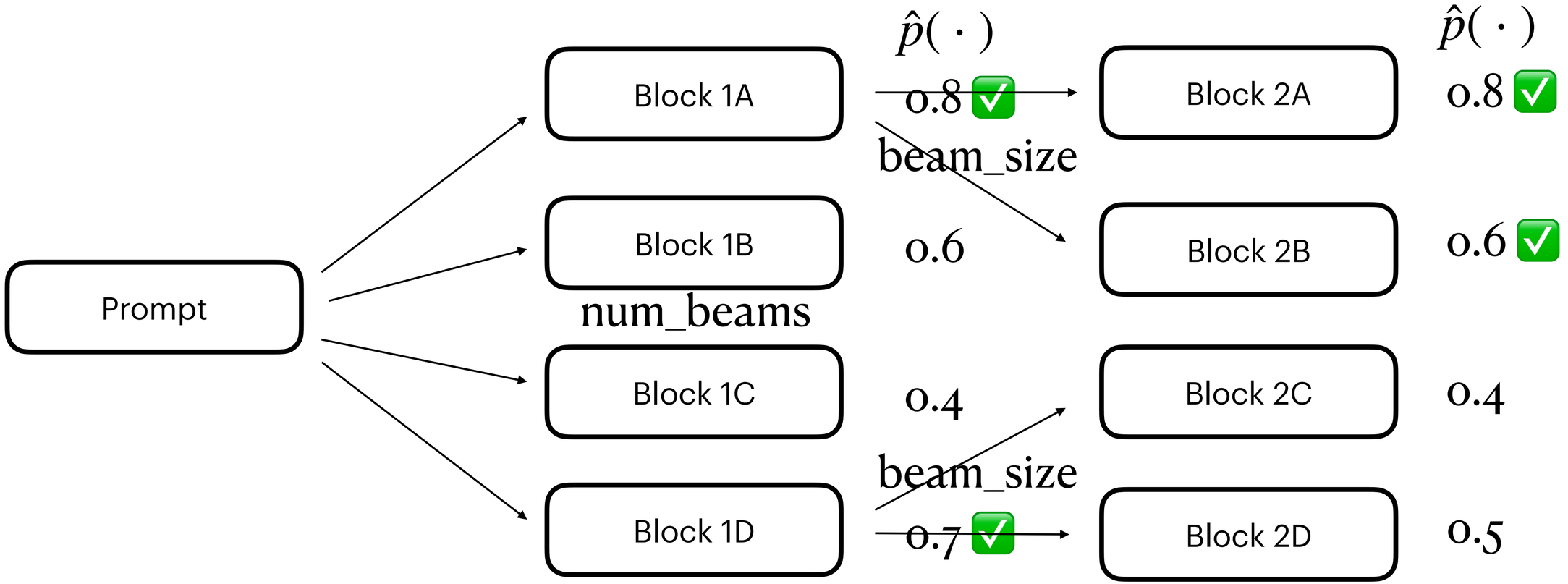


Two Benefits:

Clearer signal on *difference* between quality of generations

Faster computation, fewer value-model calls

Beam Search



Take a weighted majority vote (WMV) of the final beams

Scaling up Value Modeling for Reasoning

Collect a dataset of 2.5M CoT roll-in/roll-outs from DeepSeek models

- **Take OpenR1-Math** (94k math problems)
- For each problem, **rollin 14 responses** from each of DeepSeek-R1-1.5B to -32B (acts as "$\hat{\pi}$")
- **Rollout** using DeepSeek-R1-Distill-Qwen-1.5B (acts as "π^{ref}")
- Label the full response as $y \in$ {correct, incorrect, incomplete}

Value Model:

- Take DeepSeek-R1-Distill-Qwen-1.5B
- Train on above (x, y) data using classification loss (cross entropy)

DeepSeek-R1-Distill

Model Types:

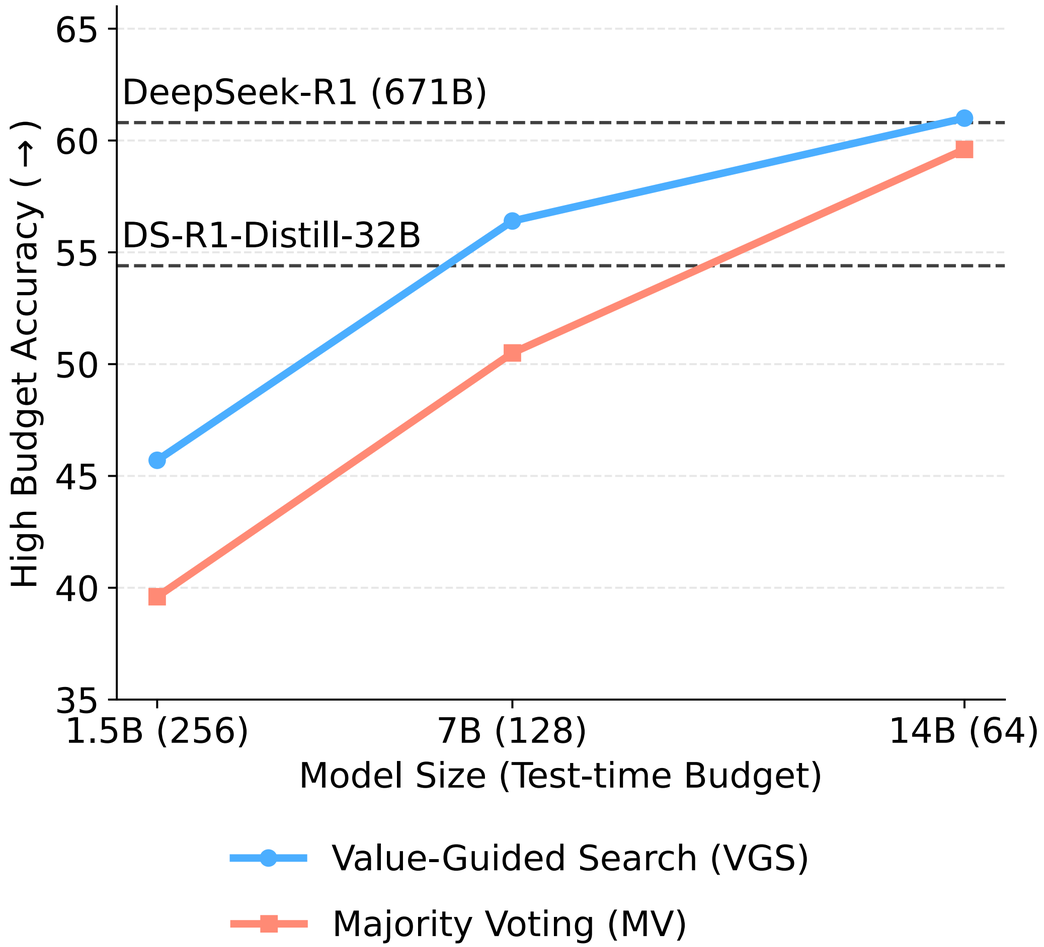
DeepSeek-R1 distilled Qwen-1.5B to 14B model

Evaluate

Competition math benchmarks (AIME 24, AIME25, HMMT Feb 24, 25)

Inference Budget

The inference budgets for 1.5B, 7B, and 14B are 256, 128, and 64 generations, respectively.



DeepSeek-R1-Distill

Model Types:

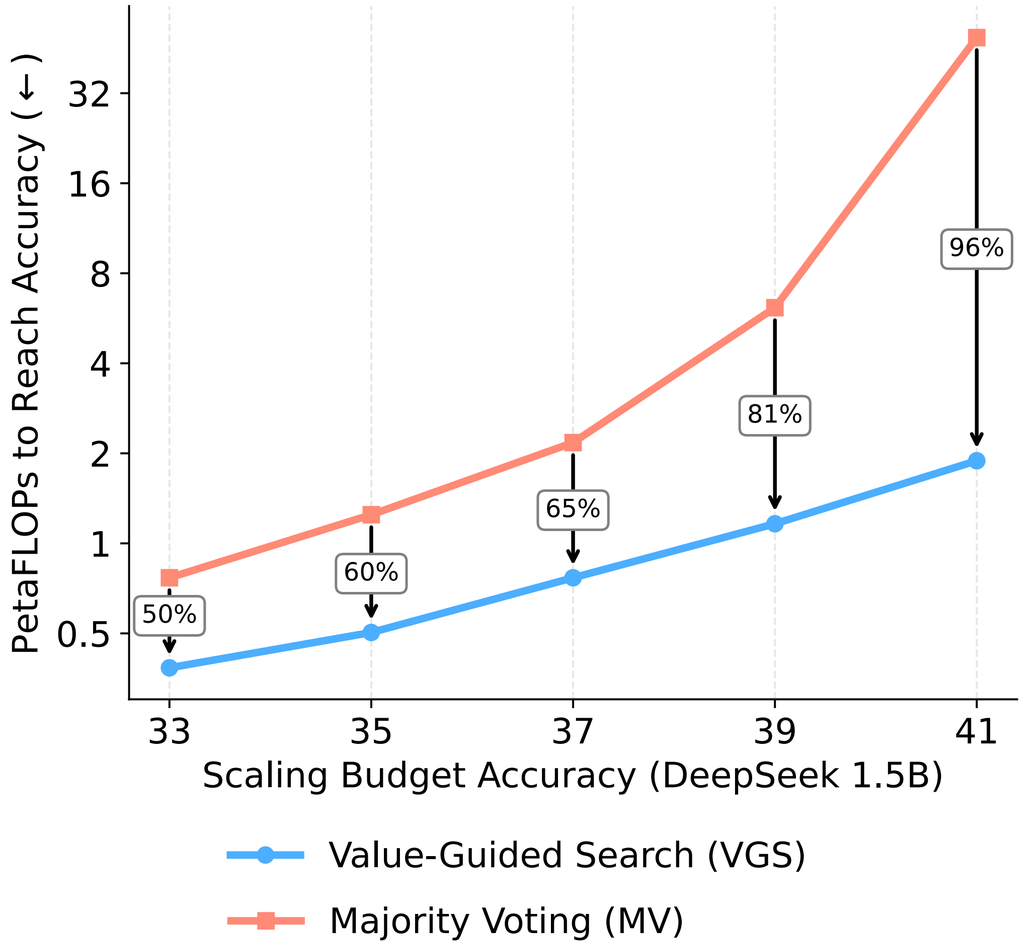
DeepSeek-R1 distilled Qwen-1.5B to 14B model

Evaluate

Competition math benchmarks (AIME 24, AIME25, HMMT Feb 24, 25)

Inference Budget

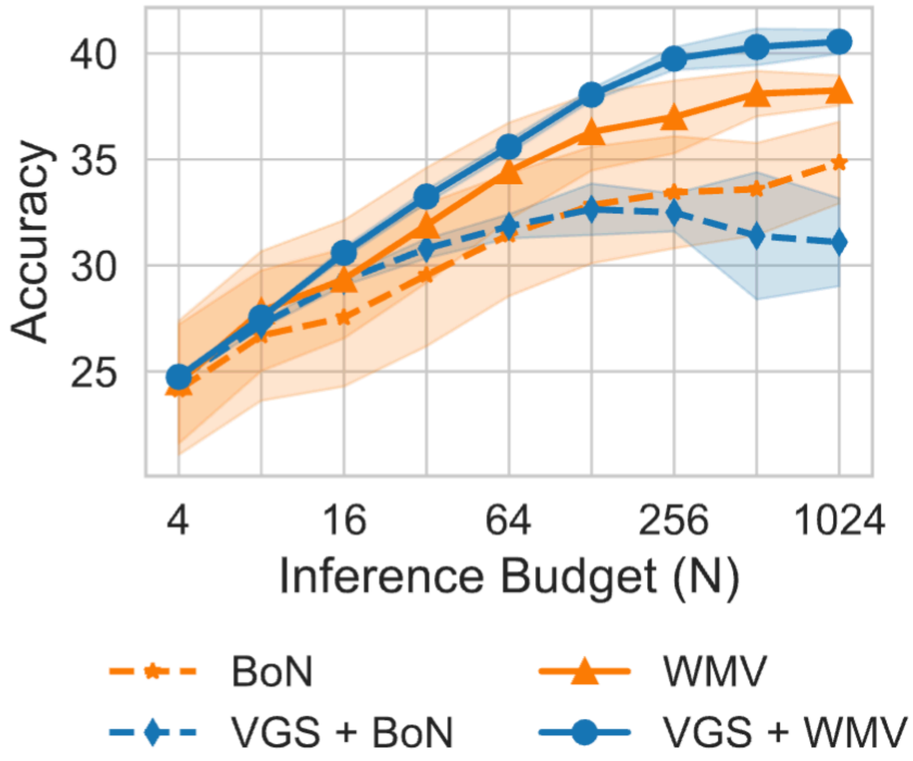
Inference computation required to reach that target accuracy.

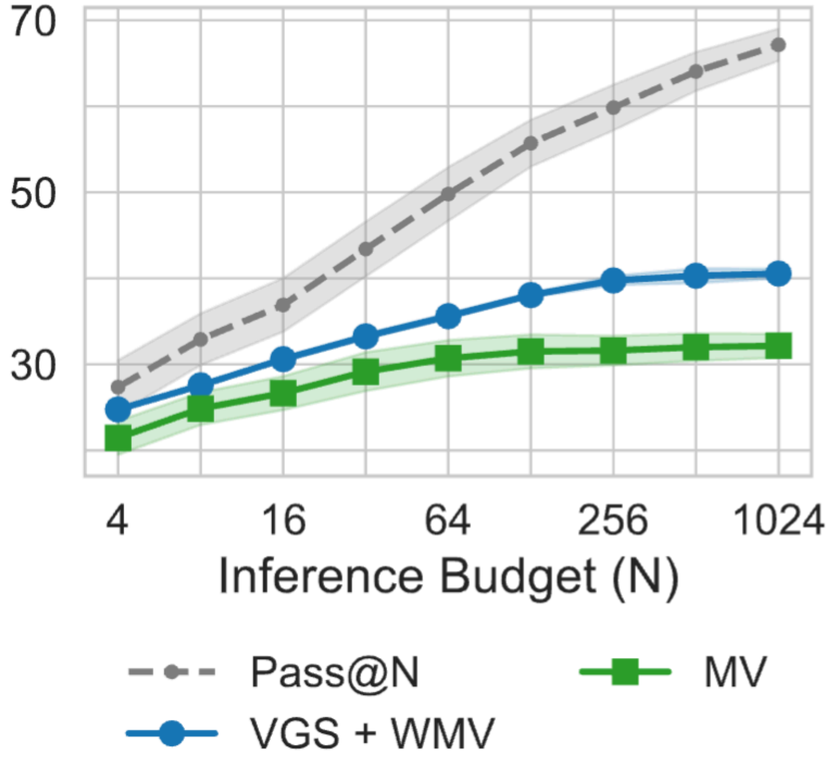


Guiding 1.5B DeepSeek

Response-Level Selection vs Search-Based Block-Level Selection

Avg of AIME '25



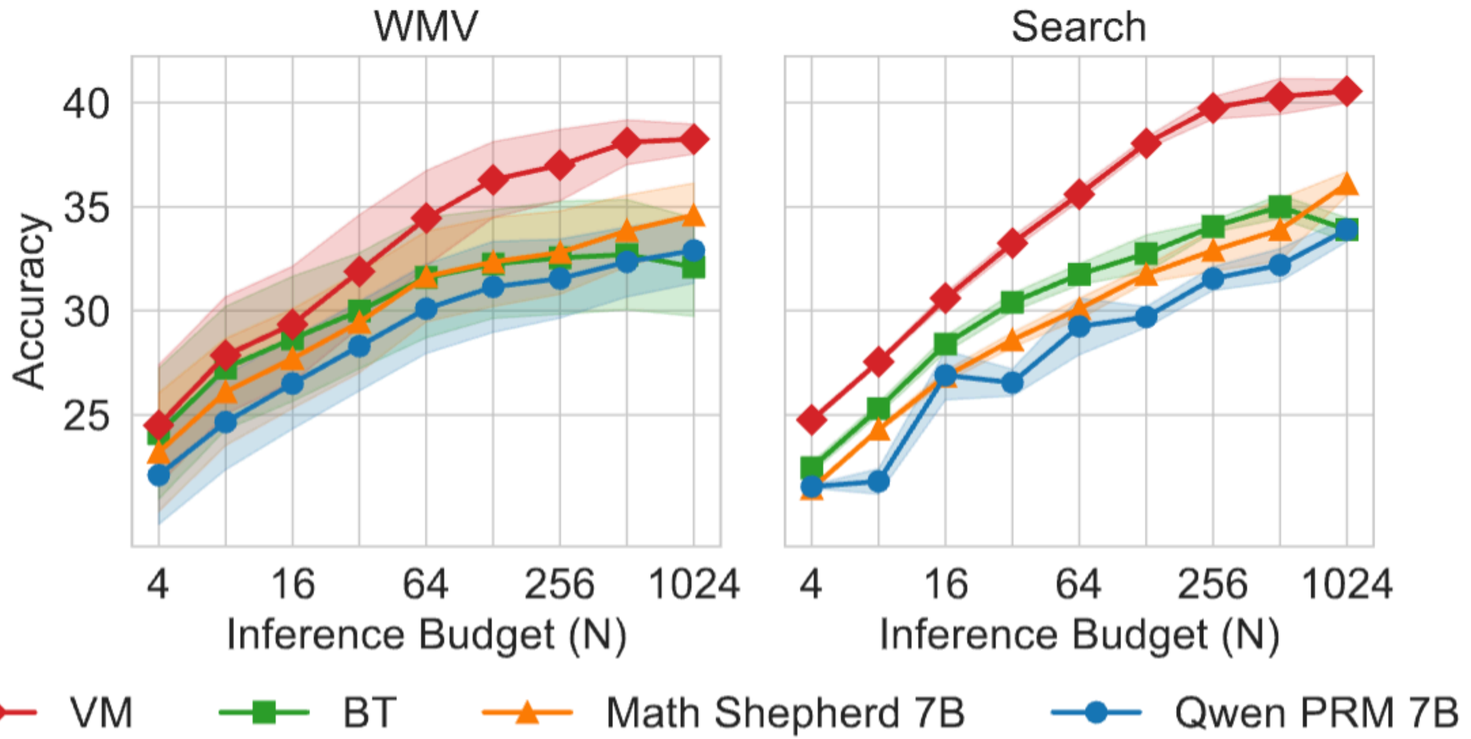


Search > WMV > BoN

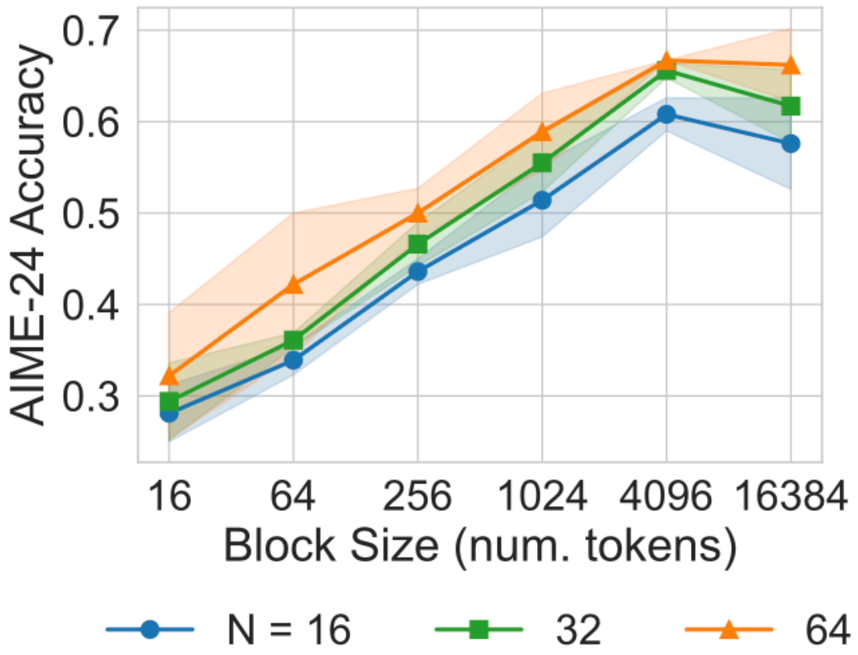
VM > no VM

Comparison to Process Reward Models

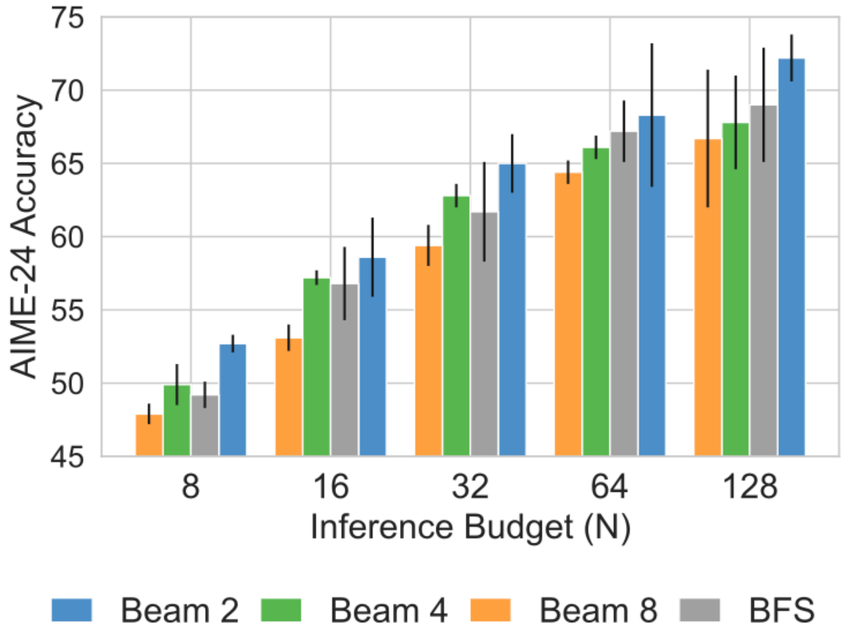
Process Reward Model (PRM) judges if current step is correct so far
Backward-looking (vs value-model being forward-looking by definition)
Requires step-level annotation, not scalable (vs only outcome-level annotation)



Ablation: Search Parameters



Optimal block size (4096) is consistent across N



Optimal beam width (2) consistent across N

LLMs implicitly execute a token-by-token MDP with a special structure.

What does the token-by-token structure of LLMs buy us?

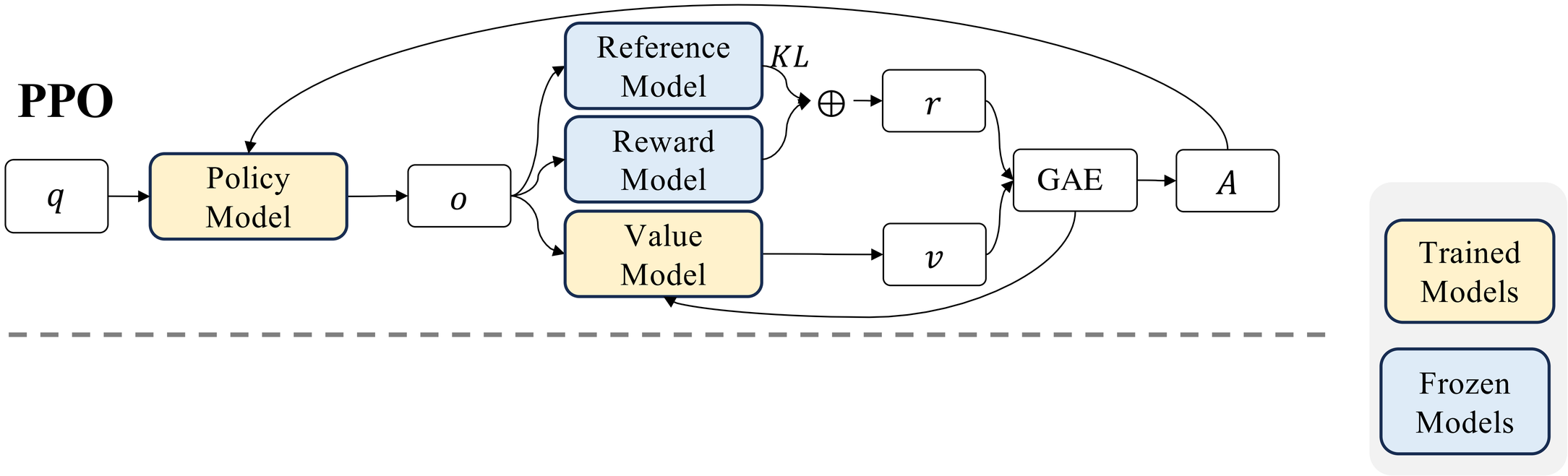
I. **Q#:** Principled value-based algorithm for LLM post-training

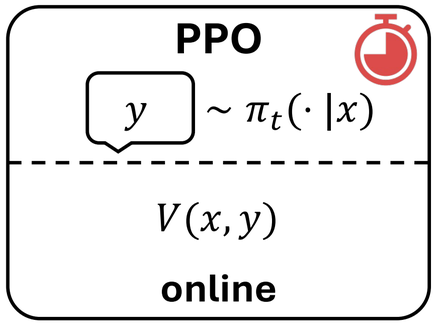
II. **VGS:** Test-time Value Guided Search

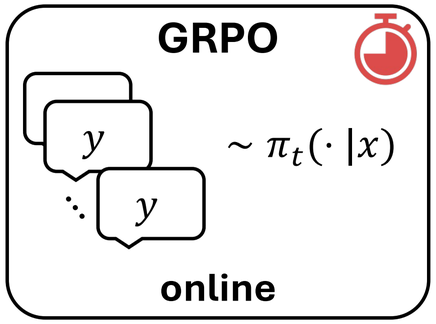
III. **A*PO**: Training-time optimization algorithm

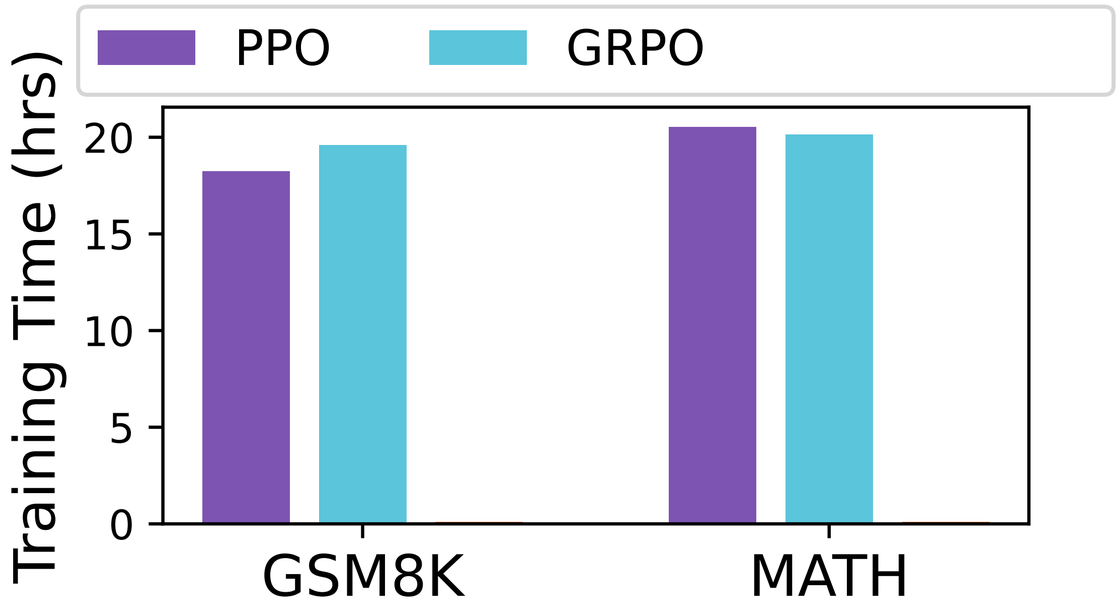
Accelerating RL for LLM Reasoning with Optimal Advantage Regression

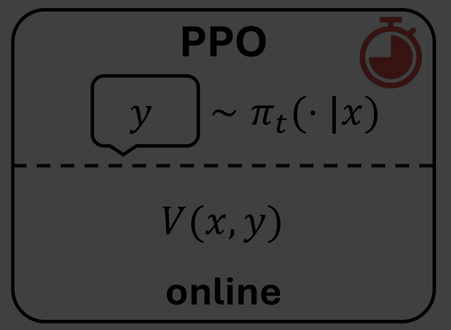
Kianté Brantley1∗, Mingyu Chen2, Zhaolin Gao3,
Jason D. Lee4, Wen Sun3, Wenhao Zhan4, Xuezhou Zhang2
1Harvard University, 2Boston University, 3Cornell University, 4Princeton University

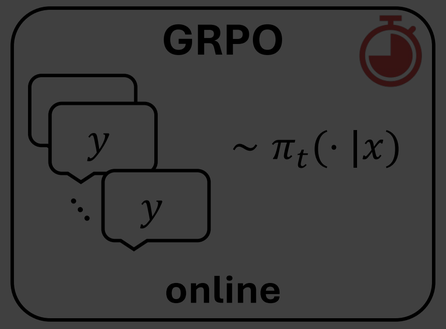

PPO
q
Policy Model
o
Reference Model
Reward Model
Value Model
KL
$\oplus$
r
v
GAE
A
Trained Models
Frozen Models


PPO
y
$\sim \pi_t(\cdot|x)$
$V(x, y)$
online


GRPO
y
y
$\sim \pi_t(\cdot|x)$
online

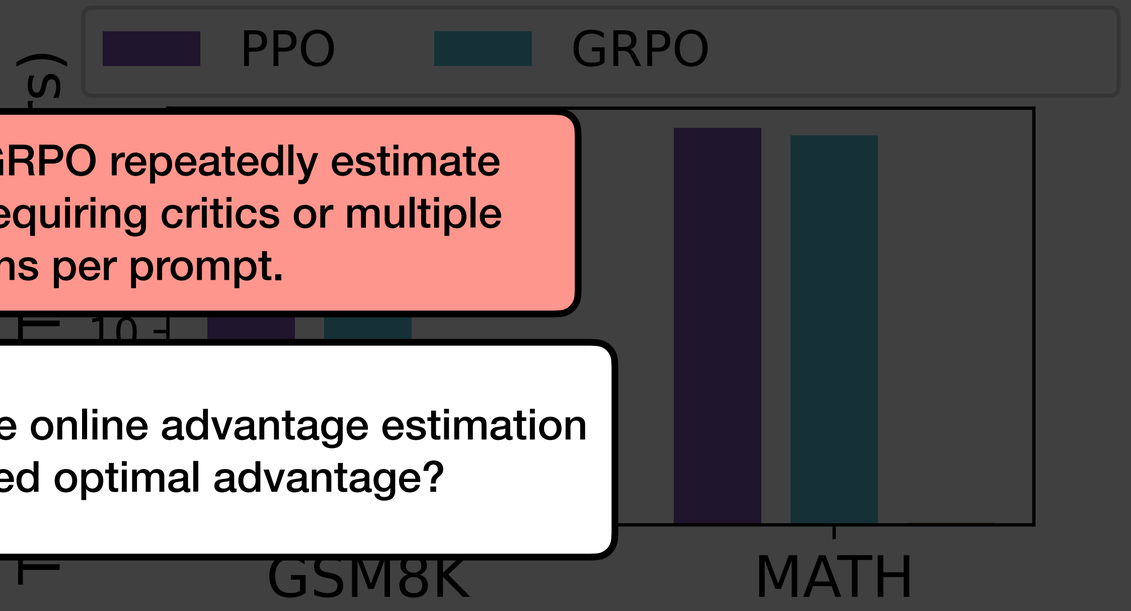

PPO
GRPO
Training Time (hrs)
0
5
10
15
20
GSM8K
MATH





Problem: PPO and GRPO repeatedly estimate advantages online, requiring critics or multiple generations per prompt.

Question: Can we replace online advantage estimation with a precomputed optimal advantage?



MinxEnt RL

$$\max_{\pi} \mathbb{E}_{\pi}\left[r(x,y)\right] - \frac{1}{\eta}\mathbb{D}_{\mathrm{KL}}(\pi \,\|\, \pi_{\mathrm{ref}})$$

Objective

$$\pi^*(y \mid x) = \frac{\pi_{\mathrm{ref}}(y \mid x)\, \exp(\eta r(x,y))}{Z(x)}, \qquad V^*(x) = \frac{1}{\eta}\log Z(x) = \frac{1}{\eta}\log \mathbb{E}_{\pi_{\mathrm{ref}}}[(y' \mid x)\, \exp(\eta r(x,y'))],$$

$$\pi^*(y \mid x) = \pi_{\mathrm{ref}}(y \mid x)\, \exp(\eta[r(x,y) - V^*(x)]),$$

Optimal Policy

$$\ell_t(\pi) := \mathbb{E}_{x,y\sim\pi_t(\cdot|x)}\left[\left(\eta \ln \frac{\pi(y\,|\,x)}{\pi_{\mathrm{ref}}(y\,|\,x)} - \left(r(x,y) - \hat{V}^*\right)\right)^2\right]$$

Regression Loss

A*PO Algorithm

1. Stage 1: Estimate $V^*(x)$ for all prompts

e.g., computed offline and kept fixed

Draw N i.i.d. samples $\{y_i\}_{i=1}^{N} \sim \pi_{\text{ref}}(\cdot \mid x)$

Compute $\hat{V}^{\star}(x) \leftarrow \beta \ln\left(\frac{1}{N}\sum_{i=1}^{N} \exp\left(\frac{r(x, y_i)}{\beta}\right)\right)$

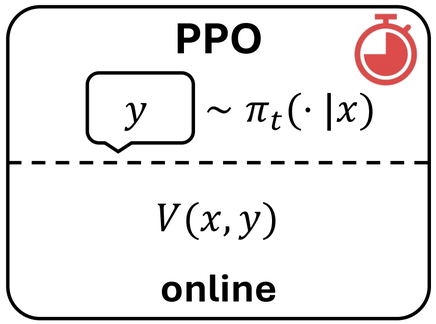
2. Stage 2 (Online): On-policy Update with one rollout per prompt

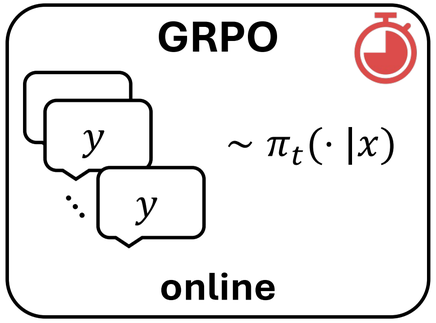
Sample batch of training samples $D_t : \{x, y\}$

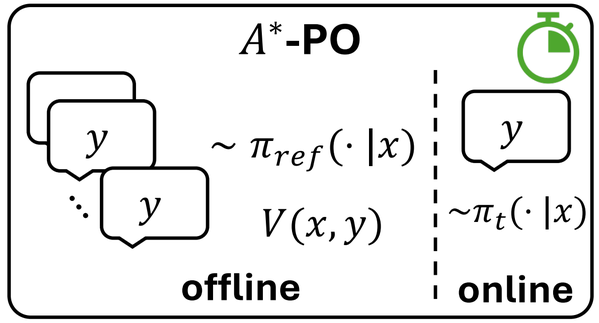
Regress to Estimated $V^*(x)$

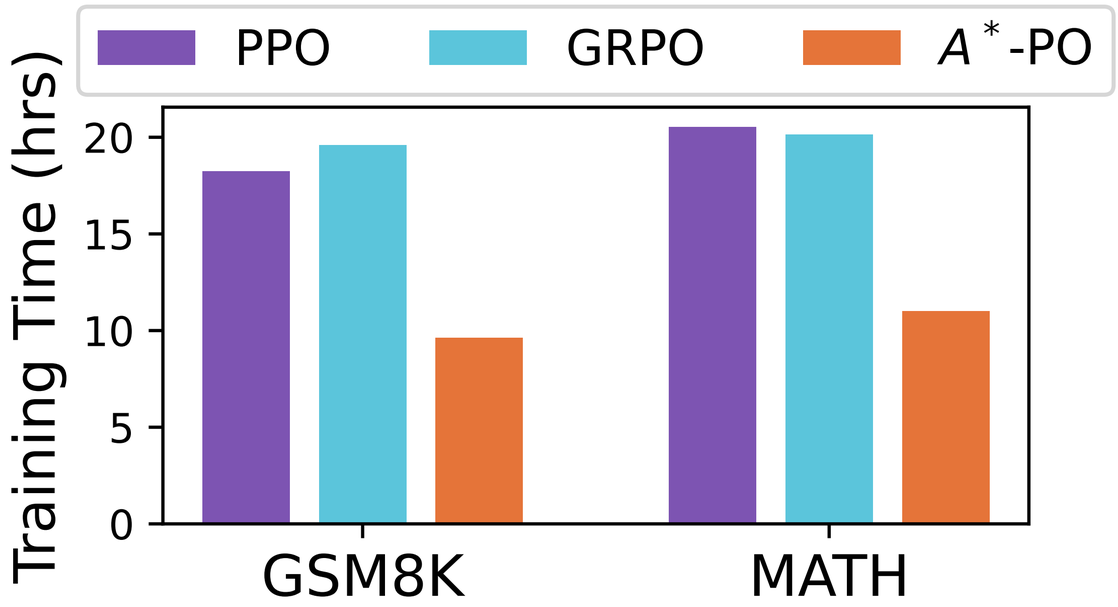
e.g., gradients only flow through π_θ online now

$$\ell(\pi_\theta) := \mathbb{E}_{x,y\sim\pi_\theta(\cdot|x)}\left[\left(\underbrace{\eta \ln \frac{\pi_\theta(y \mid x)}{\pi_{\text{ref}}(y \mid x)}}_{\text{Predictor}} - \underbrace{\left(r(x, y) - \hat{V}^*\right)}_{\text{Target}}\right)^2\right]$$


PPO
y
$\sim \pi_t(\cdot|x)$
$V(x,y)$
online


GRPO
y
y
$\sim \pi_t(\cdot|x)$
online


A^*-PO
y
y
$\sim \pi_{ref}(\cdot|x)$
$V(x,y)$
offline
y
$\sim \pi_t(\cdot|x)$
online


PPO
GRPO
A^*-PO
Training Time (hrs)
0
5
10
15
20
GSM8K
MATH

Experiments: Advanced Reasoning on MATH

MATH dataset [Hendrycks et al.,2021]:
Consists of challenging problems from high school math competitions.

Model Types:
Pre-trained Qwen2.5 models

Model Size	Method	Time (hrs)	Memory (%)	KL $(\pi \| \| \pi_{\text{ref}})$	MATH500
1.5B	*base*	/	/	/	45.8
	PPO	10.84	42.64	0.151	57.0
	GRPO	15.01	<u>42.19</u>	<u>0.091</u>	**58.0**
	REBEL	<u>10.33</u>	63.34	0.098	<u>57.8</u>
	$A^\star$-PO	**6.92**	**41.59**	**0.069**	<u>57.8</u>
3B	*base*	/	/	/	50.8
	PPO	13.59	54.95	0.111	65.8
	GRPO	18.26	<u>49.75</u>	0.102	66.0
	REBEL	<u>12.88</u>	74.32	<u>0.099</u>	**67.0**
	$A^\star$-PO	**8.78**	**49.28**	**0.082**	<u>66.2</u>
7B	*base*	/	/	/	62.8
	PPO	20.53	92.81	0.133	74.4
	GRPO	20.15	<u>77.82</u>	0.172	73.2
	REBEL	<u>14.67</u>	98.77	<u>0.124</u>	<u>74.6</u>
	$A^\star$-PO	**11.01**	**76.57**	**0.078**	**76.2**

Experiments: Long-Context Reasoning

OpenR1-Cleaned Dataset [From VGS]:

Randomly selected subset of 5,000 problems

Evaluate:

AIME (American Invitational Mathematics Examination) an annual US high school math competition

Model Types:

DeepSeek-R1 distilled Qwen-1.5B model

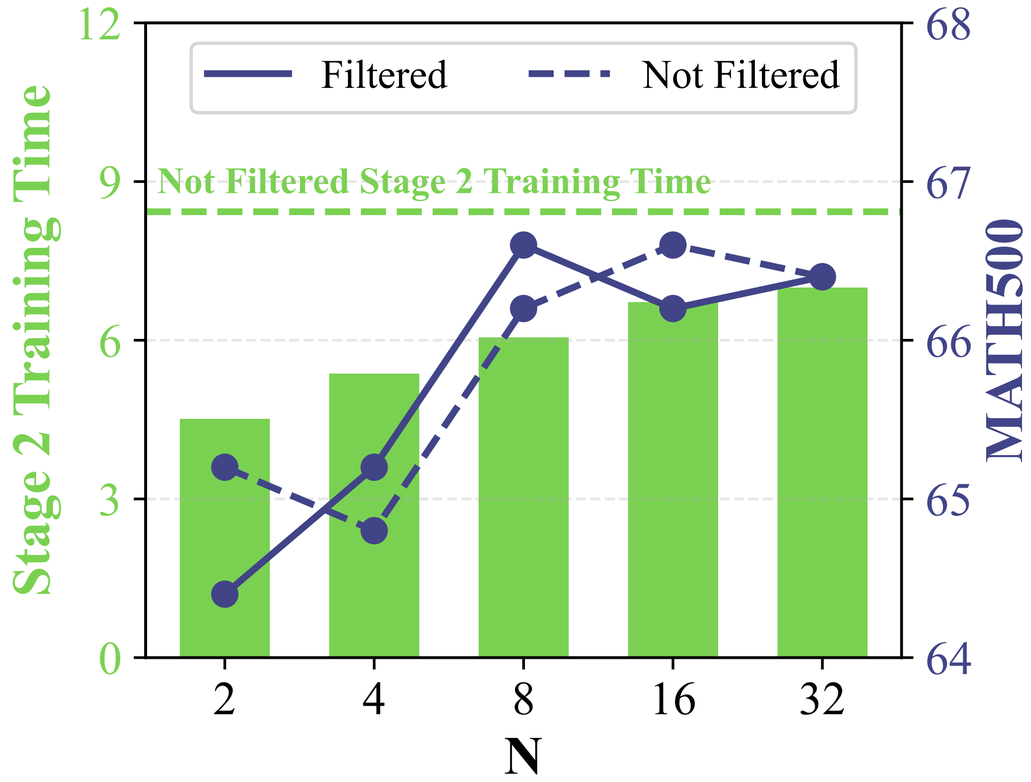
Method	AIME 24		AIME 25	
	Avg@32	Pass@32	Avg@32	Pass@32
base	21.25	60.00	21.15	43.33
PPO	**30.94**	**80.00**	25.12	**46.67**
GRPO	30.00	73.33	25.00	43.33
$A^\star$-PO	29.17	70.00	**26.67**	**46.67**

Ablation: Filtering Out Hard Problems.

Can training efficiency improve to filter out examples whose pass@N is zero?

Filter Pass@N can be done at stage 1

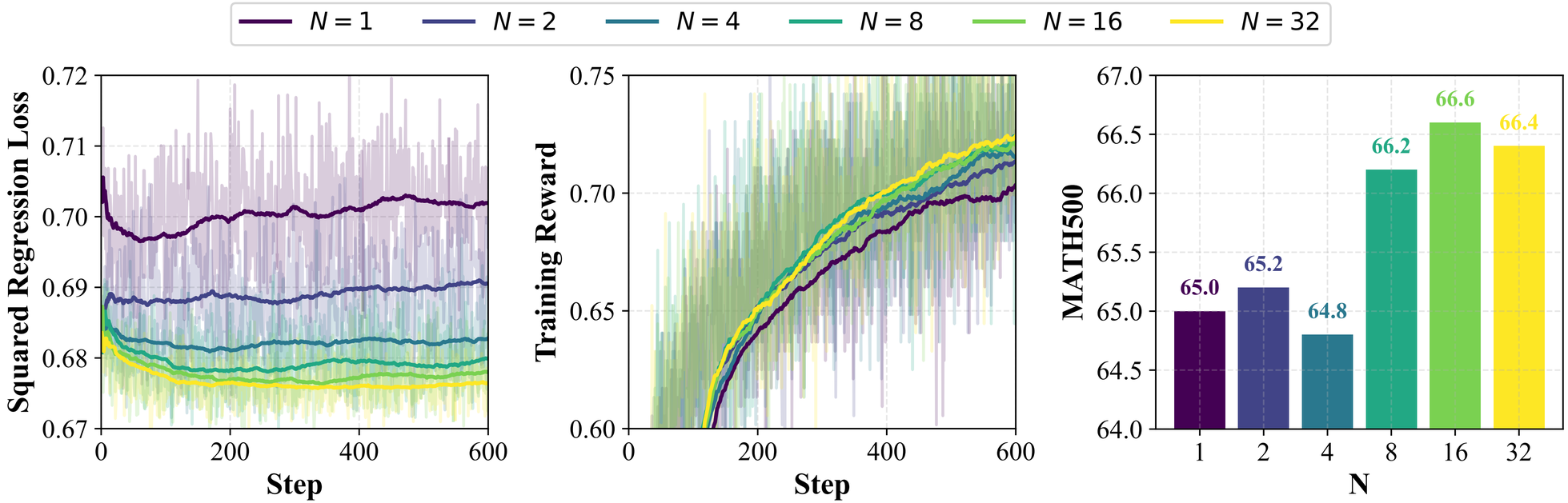
Problems that the reference policy is unable to solve are unlikely to be solved through RL post-training.



Ablation: Number of Generations

How many generations are used during stage 1 to estimate V^* on the MATH dataset?

We vary the number of generations N from 1 to 32.



LLMs implicitly execute a token-by-token MDP with a special structure.

What does the token-by-token structure of LLMs buy us?

Optimal Value *Function Guidance*

I. **Q#:** Principled value-based algorithm for LLM post — Act

II. **VGS:** Test-time Value Guided Search — Reason

III. **A*PO**: Training-time optimization algorithm — Train

Acknowledgments











